



Penny Wise and Pound Foolish!

We often try to use a smaller data type to save space. Though it looks like clever programming, it can cause nasty bugs. Let's look at an example that demonstrates this.

A few years back, I received my car insurance renewal notice. I was surprised to see a software bug in that notice! Check the figure (personal details and company name are hidden). Can you explain what could have gone wrong?

As highlighted in the image with a light red box, the 'Customer ID' entry reads `1.00E+11`, which is absurd! How could this have happened? To answer that, we'll first discuss the seemingly unrelated topic of using `printf` format specifiers in C.

In C format specifiers, when we use `%f` (fixed precision format specifier), it will print the number in decimal format (for example, 123.45). However, if the floating point number is big, it can end up printing a lengthy sequence of digits (for example, 12345678912345.67), which is difficult to read. When we use `%e` (scientific notation), it prints the floating value in the exponent format (for example, the value 123.45 will be printed as 1.234500e+02). End-users are not familiar with this scientific output format, hence using it by default is not preferable. So, what is the solution?

Fortunately, there is another format specifier, `%g`, which mixes both these approaches: It uses the decimal format for small numbers, and exponent format for large numbers. For example, if the floating point value is 123.456, `%g` will print it as 123.456, but if the floating value is 1234567.8, then it is printed as 1.23457e+06. If we use `%G`, we get the symbol "e" printed in uppercase: 1.23457E+06.

As you can see, `%g` (or `%G`) is a convenient format specifier to use, and that is why it is preferred in most software applications. In most other languages (such as Java), this approach (the exact details might vary slightly) is used by default.

Coming back to the insurance notice, can you now explain how `1.00E+11` might have got printed? I don't know what programming language

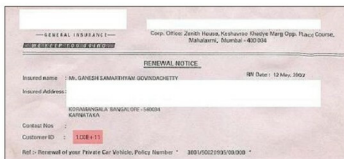


Figure 1: A bug in my insurance renewal notice

was used to develop that insurance software, nor do I have access to its source code. However, with this background about format specifiers, we can make an educated guess. The customer ID, in this case, is a number of a few digits. The programmer who wrote the code for the automation of the insurance work-flow might have been stingy in using memory space. So, instead of using a string representation for customer ID, which takes a number of characters for each ID, he (or she!) could have chosen a floating-point representation to store the number, which takes only 4 or 8 bytes, depending on whether it is a *float* or *double* data type. During testing, the tester might have used smaller customer IDs, so the numbers would have been displayed correctly. However, in actual use of the software, when the customer ID probably became a large number, the floating point number is printed in exponential form—a bug!

As you can see, trying to save a few bytes of memory per customer ID manifested itself as a bug in the software. The lesson is, be careful in trying to optimise storage space. 

By: S.G. Ganesh

The author works for Siemens (Corporate Technology). He is the author of the bestseller 'Cracking the C, C++ and Java Interview' published by Tata McGraw-Hill. You can reach him at ssganesh@gmail.com.